MimarAi Master Reference

Consolidated Plan: Idea, Full Flow, Architecture & Verification Logic

Single-source reference — combines the Interview Pipeline, MCP Logic, Deployment Plan, and Drift-Check Logic documents

- 0. What MimarAi Is (Read This First)
- 1. High-Level User Journey (End-to-End)
- 2. The Generation Pipeline (Interview → Plan)
  - 2.1 Interview Flow
  - 2.2 Feature Generation — Step by Step
  - 2.3 User Review & Control (Features)
    - 2.3.1 Comment System (used identically for Features, Structure, and Diagram sections)
    - 2.3.2 Processing a Comment Batch
    - 2.3.3 Repeatable Review Cycle
  - 2.4 Dependency-Order Generation (Backend Only)
- 3. Project Structure Generation
  - 3.1 Generation Method (same dual-independent-draft pattern)
  - 3.2 Groq's Structure-Specific Validation Checks
  - 3.3 User Review of Structure
  - 3.4 Merge & Output Format
  - 3.5 After Structure Acceptance
- 4. Diagram Generation (System Architecture & Database ERD)
  - 4.1 Trigger
  - 4.2 Generation (same dual-independent-draft pattern)
  - 4.3 Groq's Schema/Architecture Validation Checks
  - 4.4 User Review (Before Rendering)
  - 4.5 Rendering
  - 4.6 Post-Lock Changes — "Ask AI" Chat Interface
  - 4.7 Accepting the Diagram
- 5. Final Review & Sending to the Coding Agent
  - 5.1 Handling Unresolved Issues
  - 5.2 Latency & Progress UX
- 6. Getting the Plan to the Coding Agent: MCP + PDF
  - 6.1 MCP Server — What It Exposes
  - 6.2 Per-Project Authentication
  - 6.3 Rate Limiting
  - 6.4 Per-Agent Config Delivery
  - 6.5 Drift-Check Trigger via MCP
  - 6.6 Parallel Output Path — PDF
    - PDF Export Details
- 7. Drift-Check Logic (Post-Implementation Verification)
  - 7.1 Foundations the Drift-Checker Reads From
  - 7.2 Structural Pre-Scan — Avoiding False Noise
  - 7.3 Deep Verification — Hybrid Method
  - 7.4 False-Positive Protection — Dual-Model Verification
  - 7.5 Trigger
  - 7.6 Output — Per-Feature Status Labels
  - 7.7 Full Drift-Check Flow
- 8. Deployment & Infrastructure
  - 8.1 Component-to-Platform Mapping
  - 8.2 Keeping the Backend Alive (Render Free Tier)
  - 8.3 MCP Server Hosting
  - 8.4 Environment Variables & Secrets
  - 8.5 Domains & Routing

8.6 Free-Tier Cost Check
9. Cross-Cutting Design Principles (Why It's Built This Way)

0. What MimarAi Is (Read This First)

MimarAi is a planning-and-verification layer that sits **in front of** AI coding agents (like Claude Code, Cursor, or Antigravity). Instead of a student/user jumping straight into a coding agent with a vague idea, MimarAi:

1. **Interviews** the user about their idea, desired features, and fixed technical inputs (stack, database, deployment target, APIs).
2. Runs that idea through a **dual-AI generation pipeline** (Gemini + Groq, used independently of each other to cross-check) to produce a feature list, a file/folder **structure map**, and a **database/architecture diagram** — all reviewable and editable by the user.
3. Once the user accepts everything, the **complete implementation plan** (features + structure JSON + schema/architecture JSON) is handed to a coding agent — either as a downloadable PDF or via a **live MCP server connection**.
4. After the coding agent writes code, MimarAi can run a **Drift-Check**: comparing what was actually built against the accepted plan, to catch missing, broken, or divergent implementation — again using the dual-AI (Gemini + Groq) cross-check pattern to avoid false positives.

The unifying design principle across every stage: **two independent AI drafts, diffed against each other, with a capped single fix-cycle** — never an open-ended AI loop, and never a single model grading its own work.

The whole system is designed to run on **free-tier infrastructure** (Vercel, Render, Supabase, Gemini Flash + Groq), with the only paid trigger being future scale.

---

1. High-Level User Journey (End-to-End)

```
User idea + fixed inputs (stack, DB, deployment, APIs)
        │
        ▼
GENERATION PIPELINE (dual-AI: Gemini + Groq) → Feature List
        │
        ▼
USER REVIEW of features (accept / reject AI-added / comment) — repeatable cycle
        │
        ▼
USER ACCEPTS FEATURE LIST
        │
        ▼
Dependency-order generated (backend-only, no version labels shown to user)
        │
        ▼
STRUCTURE GENERATION (dual-AI) → file/folder JSON tree mapping files → features
        │
        ▼
USER REVIEW of structure (comment cycle)
        │
        ▼
STRUCTURE ACCEPTED → merged into implementation plan
        │
        ▼
DIAGRAM GENERATION (dual-AI) → DB schema JSON + architecture JSON
        │
        ▼
USER REVIEWS JSON → diagrams rendered (Mermaid.js: ERD + Architecture)
        │        "Ask AI" chat available for edits (cosmetic vs structural-conflict)
        ▼
```

```
USER CLICKS "ACCEPT THE DIAGRAM" → merged into implementation plan
        │
        ▼
FULL IMPLEMENTATION PLAN SHOWN (Features + Structure + Diagram, sectioned)
        │
   ┌────┴─────┐
   │"Update"  │ "Send to Agent"
   ▼          ▼
 (loop back   FILE SENT TO CLAUDE (coding agent)
  to review)      │
                  ▼
       Student connects via MCP (live, per-project token)
       OR uses the downloadable PDF (static, standalone)
                  │
                  ▼
       Coding agent implements the plan
                  │
                  ▼
       DRIFT-CHECK (manual click, or agent-triggered via MCP)
       → dual-AI (Gemini + Groq) verification against structure map
         + schema/architecture JSON
                  │
                  ▼
       Status-labeled Drift Report shown to user
       (✅ Implemented / ⚠ Adjusted—Please Review / ✖ Missing / ○ Not Started)
```

---

2. The Generation Pipeline (Interview → Plan)

2.1 Interview Flow

1. User provides their idea and any features they already know they want.
2. User provides fixed inputs: tech stack, database, deployment target, APIs, etc.
3. The Generation Pipeline runs (below).
4. User reviews the generated plan.
5. User accepts the final plan.
6. The accepted file is sent to Claude (the coding agent) for implementation.

2.2 Feature Generation — Step by Step

Step	What Happens
1. Parallel Independent Drafting	Gemini drafts a feature list + constraints. **At the same time**, Groq independently drafts its own feature list from the same inputs, without seeing Gemini's draft. Runs in parallel — total time = the slower of the two. Independent drafting (not "Groq reviews Gemini") matters because a model reviewing its own draft only catches errors, not *omissions*. Two independent drafts catch gaps too.
2. Diff & Merge	Feature present in Groq's list but missing from Gemini's is added into the merged list (e.g. catches something like audit logging that Gemini alone might skip).
3. First Validation (Groq)	Groq checks the merged list for: constraint violations (conflicts with stack/DB/deployment), scope relevance (is this feature actually in scope?), completeness (does every feature have constraints/exceptions defined?).
4. Single Fix Pass (Gemini)	If Groq flags a flaw, Gemini fixes it based on Groq's reason — **once only**, not a loop. Groq never fixes directly; it stays checker-only.
5. Final Validation (Groq)	Groq re-checks the fixed plan one more time. Whatever

	comes out moves forward, **even if imperfect** — no further looping. This caps total AI calls so latency stays bounded.
6. Plan Shown to User	The finalized plan (after at most one fix cycle) is presented for review.

This fixed-call-count pattern (2 parallel drafts → validate → 1 fix → re-validate) is reused **identically** for Structure Generation and Diagram Generation later — same shape, different content.

2.3 User Review & Control (Features)

- **AI-added features** (not originally mentioned by the user) are clearly labeled as AI-added, and can be:
  - **Rejected** — removed entirely, no further processing, or
  - **Commented** — user requests a change/clarification.
- **User-originated features** (typed by the user) only get **Comment** — not outright rejectable, since the user asked for them directly.

2.3.1 Comment System (used identically for Features, Structure, and Diagram sections)

- **Trigger:** user selects/highlights a feature or line; a comment icon appears.
- **Add comment:** clicking opens an inline textbox ("Leave a comment"); user types and confirms with "Add Comment."
- **Inline visibility:** comment appears immediately next to that specific line (margin/side-comment style) — no separate panel needed.
- **Accumulation, not immediate processing:** comments collect; user can comment on multiple items before submitting.
- **Counter badges:** "Review" and "Proceed" controls show a live pending-comment count (e.g. "Proceed · 3").
- **Optional Review step:** user can open "Review" to see all pending comments together, with an optional overall message.
- **Batch submit:** tapping "Proceed" sends **all** accumulated comments together in a single batch to Gemini — not one-by-one — keeping latency to one processing cycle regardless of comment count.
- Reject button is **exclusive to AI-added features**; comment logic itself is identical across all feature types.

2.3.2 Processing a Comment Batch

1. Gemini processes all submitted comments together and updates the plan.
2. Groq validates the updated plan (same propose → validate → fix pattern reused).
3. Updated plan shown back to the user, same comment/reject controls available again.

2.3.3 Repeatable Review Cycle

This comment → fix → validate → review cycle **repeats as many times as the user wants**, until they explicitly accept the plan. No cap on how many *rounds* of review — only the *fix pass within each round* is capped at one.

2.4 Dependency-Order Generation (Backend Only)

Immediately after the user accepts the feature list — and *before* structure generation begins — the system generates an internal dependency order among accepted features (e.g. Role-Based Access must exist before Loan Approval, since Loan Approval depends on it).

- Gemini proposes the order; Groq validates it.
- **Never shown to the user as version labels** (no "V1/V2" anywhere in the UI).
- Purely backend — exists so later stages (structure generation, and the drift-checker) know which features to expect before others.

Only after the user explicitly accepts the plan does the file move toward Claude.

---

3. Project Structure Generation

Happens **strictly after** the feature list is locked (never alongside it), so structure is always built against an already-finalized feature set — no risk of drifting out of sync from later feature comments/rejections.

3.1 Generation Method (same dual-independent-draft pattern)

- Gemini drafts a proposed project structure (folders/files) and maps each feature to the file(s) implementing it.
- Groq independently drafts its own structure proposal from the same finalized feature list, without seeing Gemini's draft.
- The two are diffed; conflicting file/feature assignments are surfaced for review, but **Gemini's draft wins** — Groq stays checker/second-opinion only.

3.2 Groq's Structure-Specific Validation Checks

- **Convention-adherence** — does the structure follow the standard pattern for the chosen stack (e.g. FastAPI's routes/, models/, schemas/ layout)?
- **Completeness** — is every accepted feature mapped to at least one file? No orphans.
- **No conflicting assignments** — no two features forced into contradictory file responsibilities.

3.3 User Review of Structure

Shown as its own review step (separate from feature review), using the same comment mechanism as features.

3.4 Merge & Output Format

Once accepted, structure merges into the implementation plan as a distinct section, stored as a **machine-readable JSON tree** mapping file paths → feature(s):

```
{
  "structure": {
    "src/routes/loan.py": ["Loan & Interest Management"],
    "src/models/loan.py": ["Loan & Interest Management"],
    "src/routes/auth.py": ["Role-Based Access Control"],
    "src/models/audit_log.py": ["Audit Logging"]
  }
}
```

This JSON exists specifically for **machine verification** later (the Drift-Checker parses it directly). The user-facing view still renders it as a readable folder/file tree.

3.5 After Structure Acceptance

Plan moves to **Diagram Generation** — final approval and sending to Claude now happens only after diagrams are *also* accepted, not immediately after structure.

---

4. Diagram Generation (System Architecture & Database ERD)

4.1 Trigger

Starts automatically right after structure is locked — no new questions asked; all needed data (features, constraints, tech stack, structure) is already available.

4.2 Generation (same dual-independent-draft pattern)

Gemini drafts, from locked features + structure + fixed inputs: - **Database schema JSON** — tables, columns, data types, primary/foreign keys, relationships. - **Architecture JSON** — services/layers, protocols (HTTPS, gRPC, WebSocket), ports.

Groq independently drafts its own version of both, without seeing Gemini's draft. Diffed and merged; conflicts surfaced but **Gemini's draft wins** (Groq stays checker).

4.3 Groq's Schema/Architecture Validation Checks

- **Referential integrity** — every foreign key links to a valid primary key.
- **Feature coverage** — every feature's data is represented in some table.
- **Protocol/tech-stack consistency** — architecture protocols match the chosen stack.
- **No conflicting dependencies** — services don't depend on each other illogically.

If a flaw is found: Gemini gets a single fix pass, Groq re-validates — same capped one-cycle pattern as everywhere else.

4.4 User Review (Before Rendering)

Clean JSON shown to the user before any diagram is drawn. Comment mechanism available; **no "Reject" option** here (whole schema is AI-generated) — only Comment/Adjust.

4.5 Rendering

Accepted JSON passed to a **separate, deterministic rendering engine (Mermaid.js)**. The AI never draws the diagram itself — it only produces structured data; rendering is purely mechanical, eliminating layout-hallucination risk. Two diagrams produced: **Database ERD** and **System Architecture Diagram**.

4.6 Post-Lock Changes — "Ask AI" Chat Interface

User can describe a change in plain language on the diagram screen. Every request is classified as:

- **(a) Cosmetic change** (renaming, data type, index) — applied directly: Gemini fixes, Groq does a lightweight check.
- **(b) Structural conflict** (affects a locked feature/structure) — **refused by default**, with a clear explanation why.
  - **Forced override:** if the user insists, system asks for explicit confirmation ("this will also update your locked specification"), then propagates the change **upstream** into the feature/structure JSON (not just the diagram), runs a lightweight targeted re-check, and labels the affected item **"Manually Amended."**
  - This propagation is **visible everywhere**, not just backend data — e.g. if a table tied to "Loan Management" is removed, that feature's description/constraint text in the Features section is also updated (never left showing stale, contradictory text), and the "Manually Amended" label appears in the Features section too — so Features, Structure, and Diagram sections never contradict each other.
- Whether cosmetic or forced, diagrams **re-render immediately**. Ask-AI edits only update the working JSON at this stage — not the implementation-plan file itself (that only happens on explicit "Accept the Diagram").

4.7 Accepting the Diagram

"Accept the Diagram" button confirms it — only then does the current JSON merge into the implementation plan (not on every individual Ask-AI edit).

---

5. Final Review & Sending to the Coding Agent

Once features, structure, and diagram are all accepted and merged, the user sees the **complete implementation plan** with two buttons:

- **“Send to Agent”** — sends the complete file directly to Claude (the coding agent).
- **“Update”** — opens a single unified scroll-view showing Features, Structure, and Diagram as visually distinct labeled sections. Comments attach to the specific section they’re made on and route to that section’s existing fix mechanism (feature Gemini-fix/Groq-validate; the structure equivalent; or diagram Ask-AI).
    - If a diagram-section comment is resolved via “Update,” it merges directly back into the plan **without** requiring a second “Accept the Diagram” click — that button only applies to the *initial* diagram-generation step, avoiding redundant friction.

No automatic timeout / auto-send. If the user takes no action, the project simply stays “Pending Review” indefinitely (a gentle reminder badge may show, but nothing progresses without explicit user choice).

5.1 Handling Unresolved Issues

Because the fix loop only runs once, a flaw can occasionally survive both validation checks. Rather than hiding this or using discouraging language (“not fully validated”), the system uses a neutral badge:

> ***“Adjusted — Please Review”***

This says “we changed something here, please confirm it’s correct” — honest and transparent, without implying the tool is unreliable. This exact label is reused later in the Drift-Check output for consistency.

5.2 Latency & Progress UX

Because every fix loop is capped at one cycle, total AI calls per stage are fixed and predictable: parallel draft → validate → fix → re-validate. During processing the user sees **real, streamed backend-stage status** (not a fake loading animation), e.g.: “Validating inputs…” → “Generating feature plan…” → “Cross-checking with second model…” → “Merging & catching missed features…” → “Validating constraints…” → “Finalizing your implementation plan…”

This (1) makes the wait feel like visible progress, and (2) visibly demonstrates the two-model validation is genuinely happening, reinforcing the quality claim rather than just stating it.

---

6. Getting the Plan to the Coding Agent: MCP + PDF

Once the plan is sent, the student can connect their coding agent to MimarAi in **two parallel ways** — live (MCP) or static (PDF). They aren’t mutually exclusive.

6.1 MCP Server — What It Exposes

The MCP server runs as part of the **Render-hosted backend** — a single shared instance serving all students’ projects. It is **passive**: it only responds when an agent calls a tool; it cannot proactively detect that code was written or push updates on its own. Two tools:

Tool	Purpose
`get_implementation_plan(project_id)`	Returns the accepted implementation plan (features + structure JSON + schema/architecture JSON) so the agent can fetch it live instead of the student copy-pasting a file.
`check_drift(project_id)`	Triggers the Drift-Check logic (structure map + schema JSON vs. actual code) and returns the status-labeled drift report.

6.2 Per-Project Authentication

- On plan acceptance, when the student chooses “Connect via MCP,” the backend generates a **unique,**

random, non-guessable token tied to that specific project.
- Token is embedded in the config snippet (URL or header) the student copies into their coding agent (Claude Code, Cursor, Antigravity, etc.).
- Every MCP tool call includes this token; the server validates it and **scopes all data access to that one project only** — no student can access another's data.
- Token is stored **hashed** in the database (never raw) — standard password-security practice.
- **"Regenerate Token"** option available if a token is leaked. No automatic expiry for now (mirrors GitHub personal access tokens).

6.3 Rate Limiting

- Applied **per-token**, not per-IP (a token reliably identifies a project regardless of network changes).
- Roughly **20–30 requests/hour** per token — generous relative to real usage (fetching the plan, occasional drift-checks), but enough to catch accidental loops or abuse.
- Over the limit → server returns **HTTP 429** with a clear message ("Rate limit exceeded, try again in X minutes") rather than failing silently.
- Tracking: in-memory or Redis counter if available, or a simple DB-level counter table (token, count, last-reset-time) as a free-tier-friendly fallback.
- This guards against the **"noisy neighbor" problem** — since the MCP server instance is shared, one project's malfunctioning/looping agent could otherwise degrade service for everyone else.

6.4 Per-Agent Config Delivery

Different coding agents expect different config formats (e.g. a `.cursorrules`-style file for Cursor vs. a separate config JSON for Claude Code). MimarAi offers **"Copy Config"** buttons per agent, each embedding the same per-project token in that agent's expected format.

6.5 Drift-Check Trigger via MCP

- **Primary trigger:** the student manually clicks "Check Drift" inside the MimarAi UI — always reliable.
- **Secondary trigger:** the coding agent is instructed (via a config file like `CLAUDE.md` or `.cursorrules`) to call `check_drift()` once it finishes implementing a feature. **Best-effort only** — the passive MCP server cannot force the agent to call it; it only works if the agent follows the instruction it was given.

6.6 Parallel Output Path — PDF

Alongside MCP, the full plan is also downloadable as a **PDF** — usable standalone without MCP at all (e.g. uploaded directly into a Claude chat, exactly like this document). This gives students a choice: live MCP connection for agents that support it, or a static file for anything that doesn't.

PDF Export Details

- Generated **server-side** once the user accepts the full implementation plan.
- Contains the **full plan in JSON form** — features, constraints, structure JSON, schema/architecture JSON — not just a human-readable summary.
- **JSON only, no rendered diagram images.** Visual ERD/architecture diagrams stay UI-only; the PDF is meant to be a standalone, machine-usable plan.
- Served as a downloadable file directly from the backend; no separate hosting needed.

---

7. Drift-Check Logic (Post-Implementation Verification)

Once the coding agent has implemented some or all of the plan, MimarAi can verify the actual codebase against the accepted plan.

7.1 Foundations the Drift-Checker Reads From

Two JSON structures, both coming from the **same locked, dual-validated (Gemini + Groq) implementation plan** — so they can never drift apart from each other:

1. **Structure map** — file path → feature(s) it implements (generated at the end of the interview pipeline, Section 3.4 above).
2. **Schema/architecture JSON** — tables, columns, relationships, services, protocols, ports (generated during diagram generation, Section 4 above). Merges into the same implementation-plan file as the structure map.

Because the drift-checker reads directly from this JSON, it doesn't need to guess file locations or blindly scan the whole codebase — it already knows exactly which files should exist for which features. This is what makes the structural check fast and deterministic rather than exploratory, and it also means the drift-checker can verify **schema conformance** (does the actual DB have the expected tables/columns/relationships?), not just file/function presence.

7.2 Structural Pre-Scan — Avoiding False Noise

Before deep verification, a light scan checks (using the structure map) whether *any* trace of each feature exists in the codebase (matching expected file paths, function names, folder structure).

- A feature with **zero trace** is marked **Not Started** and skipped entirely — **not treated as an error**, since the project may simply not have reached that feature yet.
- Only features with *some* implementation present move on to deep verification.

This prevents the drift-checker from generating noisy, discouraging reports on a project that is intentionally still in progress.

7.3 Deep Verification — Hybrid Method

For every feature that passed the pre-scan, four checks run, escalating in how much they rely on an LLM:

Check	Method
Structural check	Confirms expected files/functions/endpoints (per structure map) exist. Fast, deterministic — **no LLM**.
Schema conformance check	For DB-tied features, confirms actual DB schema matches the accepted schema JSON. Same deterministic approach — **no LLM guessing**.
Constraint check	Numeric/rule-based constraints (e.g. "interest rate must be 4%–18%") verified via pattern-matching or actual code execution — **not LLM prediction**, which is more failure-prone for exact logic.
Semantic/behavioral check	Used **only as a last resort**, where automated testing isn't practical (e.g. verifying an approval workflow's intent). The LLM reads the code and evaluates it — the fallback, not the primary method.

7.4 False-Positive Protection — Dual-Model Verification

The same dual-independent-model pattern from generation is reused here: - Gemini independently verifies the codebase → draft drift report. - Groq independently verifies the same codebase → its own report. - The two reports are diffed/merged — **only discrepancies both models agree on are confirmed drift.**

This reduces the risk of a single model hallucinating a false "missing" or "broken" result.

7.5 Trigger

- **Primary:** user manually clicks "Check Drift" in the MimarAi UI — always reliable.
- **Secondary:** coding agent instructed via config file (`CLAUDE.md` / `.cursorrules`) to call the MCP `check_drift()` tool once it finishes implementation — **best-effort only**, since the MCP server is passive and cannot proactively detect completion.

7.6 Output — Per-Feature Status Labels

Badge	Meaning
✅ **Implemented**	Structural and constraint checks both pass.
⚠ **Adjusted — Please Review**	A mismatch was found; needs manual review (same badge used in the interview/review stage, for consistency).
✕ **Missing**	Some related code exists, but an expected piece was not found.
○ **Not Started**	No trace of implementation yet — not treated as an error.

7.7 Full Drift-Check Flow

```
Structure map + schema/architecture JSON loaded (from accepted implementation plan)
        │
        ▼
Structural pre-scan (what exists at all, per the structure map?)
        │
        ▼
Filter to only "started" features, using dependency-order
        │
        ▼
Deep check each feature (Gemini):
  - Structural check (against structure map)
  - Schema conformance check (against schema/architecture JSON)
  - Constraint check
  - Semantic/behavioral check (last resort)
        │
        ▼
Groq independently re-verifies the same features
        │
        ▼
Diff / merge results — only shared findings marked "confirmed drift"
        │
        ▼
Status-labeled drift report shown to user
```

---

8. Deployment & Infrastructure

8.1 Component-to-Platform Mapping

Component	Platform	Why
Frontend (Next.js + Mermaid.js)	**Vercel** (free tier)	Frontend is stateless/static-friendly — exactly Vercel's serverless model.
Backend (FastAPI + SQLAlchemy + Pydantic)	**Render** (free tier)	Needs a persistent server — not just for API calls but also to host the MCP server, which needs long-lived connections (unlike Vercel's serverless model).
Database	**Supabase** (PostgreSQL, free tier)	Managed, no separate database hosting needed.
AI calls	**Gemini Flash + Groq**, called from backend via an LLM gateway	No separate hosting required — just API calls.

8.2 Keeping the Backend Alive (Render Free Tier)

- Render's free tier spins down after ~15 minutes of inactivity, causing a cold-start delay on the next request.
- An external keep-alive ping (cron-job.org or UptimeRobot, both free) hits the backend every **10–14 minutes** to prevent it from ever going idle.
- This is the **standing deployment approach for the whole project**, not a temporary V1-only workaround.
- A graceful loading state ("Connecting to MimarAi server…") is still shown in the UI/agent response as a fallback in case a cold-start happens anyway.

8.3 MCP Server Hosting

Runs as part of the same Render-hosted backend (shared single instance serving all students' projects) — same keep-alive-ping approach applies. Full auth/rate-limiting detail is in Section 6.2–6.3 above.

8.4 Environment Variables & Secrets

> *Context: an earlier incident found real API keys shipped inside an uploaded `.env` file in an Antigravity-built codebase — secrets must be handled carefully this time.*

- All API keys (Gemini, Groq, Supabase, JWT secret) go into **Render's and Vercel's environment-variable settings** — never committed to the repository.
- A `.env.example` file with **placeholder values only** can be committed for reference — the real `.env` never should be.

8.5 Domains & Routing

- Vercel frontend and Render backend sit on different default domains (e.g. `mimarai.vercel.app` and `mimarai-backend.onrender.com`) unless a custom domain is configured later.
- Frontend calls the backend via its Render URL — **CORS** must be configured on the backend to allow requests from the Vercel frontend's domain.

8.6 Free-Tier Cost Check

Everything stays within free tiers: Vercel (frontend), Render (backend + MCP), Supabase (database), Gemini Flash + Groq (AI) — matching the **$0-infrastructure-cost goal for the whole project**, not just a V1 phase. The only paid element would be if usage/scale outgrows free-tier limits later.

---

9. Cross-Cutting Design Principles (Why It's Built This Way)

These patterns repeat across every stage (features, structure, diagrams, drift-check) and are the core "shape" of MimarAi:

1. **Dual-independent-drafting, not sequential review.** Gemini and Groq each draft independently without seeing the other's output first, so gaps get caught (not just errors) — used in feature generation, structure generation, diagram/schema generation, and drift verification.
2. **Capped single fix-cycle.** Every "Groq validates → Gemini fixes" loop runs exactly once per submission, keeping AI-call count (and therefore latency) fixed and predictable — never an open-ended retry loop. The *user-facing review cycle* (commenting and resubmitting) can repeat indefinitely; it's only the *internal automated fix pass* that's capped.
3. **Groq never writes, only checks.** Across every stage, Groq's role is strictly validation/second-opinion; Gemini (or the deterministic engines in drift-check) does all the actual writing/fixing. This keeps one model always independent of the other's corrections.
4. **No version labels shown to the user.** Internal sequencing (dependency-order) is backend-only; the user-facing language is "Adjusted — Please Review," not "V1/V2" or "not fully validated."
5. **Deterministic over LLM-guessed wherever possible.** Structural checks, schema conformance

checks, and diagram rendering are all mechanical/deterministic — the LLM is reserved for genuinely last-resort semantic judgment calls (drift-check's behavioral check) or generative drafting.

6. **Single source of truth.** Structure JSON and schema/architecture JSON both live inside the same locked implementation-plan file, so they can't drift apart from each other — and the same file is what's downloadable as PDF or served live via MCP.
7. **Everything free-tier by default,** with explicit keep-alive workarounds for Render's cold-start behavior rather than upgrading to a paid tier.

End of master reference. This single document supersedes the need to separately read the Interview & Generation Pipeline, MCP Logic, Deployment Plan, and Drift-Check Logic documents — all content from those four is consolidated above.